Name / jméno: _____

Question:	1	2	3	4	5	6	7	8	9	10	Total
Points:	2	2	2	2	2	2	2	2	2	2	20
Score:											

Instructions / Pokyny

- Please write your name. / Prosím, napište své jméno.
- You have 45 minutes. / Máte 45 minut.
- The test is worth 20 points in total. A minimum of 10 points is required to pass. / Test je za celkem 20 bodů. Pro úspěšné složení je nutné získat minimálně 10 bodů.
- Unless otherwise stated, consider the code to be Scala 3. / Pokud není uvedeno jinak, považujte kód za Scala 3.
- Answer the questions in the spaces provided on the question sheets. / Odpovězte na otázky do připravených míst na listech se zadáním.
- For multiple-choice questions, mark **all** correct answers; there may be more than one. / U otázek s výběrem odpovědí zaškrtněte **všechny** správné odpovědi; může jich být více.

Questions / Otázky

1. [2 points] What will it print?

```scala
class A:
  def m1: Int = 1

class B extends A:
  def m2: Int = 2

class C extends B:
  def m3: Int = 3

class Container[T]:
  def store(x: T): T = x

val container = Container[B]()
val v = container.store(C())

println(v.m3)
```

2. [2 points] Instances of which classes can be passed as parameter `x` in the following function? Mark all correct answers.

```scala
def f[A >: MyClass](x: A): Unit
```

- ☐ `MyClass` only
- ☐ `MyClass` and all its superclasses
- ☐ `Int`
- ☐ `MyClass` and all its subclasses
- ☐ `String`

3. [2 points] What will it print?

```
class A:
  def m = 1

class B extends A:
  override def m = super.m + 1

class C extends B:
  override def m = super.m + 2

class D extends C:
  override def m = super.m + 3

def eval(x: A) = x.m
def eval(x: C) = x.m * 10
def eval(x: D) = x.m * 100

val obj: C = D()
println(eval(obj))
```

4. [2 points] Which type can be used for parameter `b` in class `Y`?

```
class A
class B extends A
class C extends B

class X {
  def f(b: B): B = new B
}

class Y extends X {
  override def f(b: ?): B
}
```

5. [2 points] What will it print?

```
class A:
  def ma(): Int = 1

class B extends A:
  def mb(): Int = 2

class C extends B:
  def mc(): Int = 3

class Processor[T <: B]:
  def compute(x: T): T = x

val processor = Processor[C]()
println(processor.compute(C()).mc())
```

6. [2 points] Given the definition `class Consumer[-T]`, mark all correct statements.

- ☐ `T` is a covariant type parameter
- ☐ For any classes `A` and `B`, if `A <: B` then `Consumer[A] >: Consumer[B]`
- ☐ For any classes `A` and `B`, if `A <: B` then `Consumer[A] <: Consumer[B]`
- ☐ `T` is a contravariant type parameter
- ☐ `T` is an invariant type parameter
- ☐ For any classes `A` and `B`, if `A <: B` then `Consumer[B] <: Consumer[A]`

7. [2 points] What will it print?

```
class A:
  def x = 1
  def y = x + 1
  def z = y + 2

class B extends A:
  override def x = 2

class C extends B:
  override def y = super.y * 10

def compute(obj: A) = obj.z
def compute(obj: B) = obj.z * 10
def compute(obj: C) = obj.z + 100

val v: B = C()
println(compute(v))
```

8. [2 points] Imagine a generic `class Collection[T]` representing a collection of elements of type `T`. What variance annotation is needed for the following code to compile? Mark all correct answers.

```
class Dog extends Animal

var dogs: Collection[Dog] = ...
var animals: Collection[Animal] = dogs
```

- ☐ Covariant
- ☐ Contravariant
- ☐ Invariant
- ☐ Will always compile
- ☐ Will never compile

9. [2 points] What will it print?

```
class A:
  def f(a: A): Int = 1

class B extends A:
  def f(b: B): Int = 2
  override def f(a: A): Int = 3

val x: A = B()
val y: B = B()

println(y.f(x))
```

10. [2 points] What will it print?

```
class A:
  def m: Int = 1

class B extends A:
  override def m: Int = 2

class C extends B:
  override def m: Int = 3

class Consumer[-T]:
  def accept(x: T): Int = x.m

val consumer: Consumer[C] = Consumer[B]()
println(consumer.accept(C()))
```